



LedgerWright

An AI reconciliation agent that closes the books for small businesses automatically — matching, explaining, and verifying every transaction in minutes, not hours.

PROBLEM STATEMENT

Omni_FinTech_5 — Automated Account Reconciliation for Small Businesses

TRACK

FinTech & Financial Inclusion

SUBMISSION TYPE

Problem Understanding · Solution · Architecture · Tech Stack · Implementation Plan

FORMAT

Single Consolidated PDF

CONTENTS

01 Problem Understanding

02 Proposed Solution

03 System Architecture

04 Key Features

05 Technology Stack

06 Implementation Plan

SECTION 01

Problem Understanding

Why manual account reconciliation is a slow, error-prone bottleneck for small businesses — and why it matters for financial inclusion.

The core problem

Every small business generates two independent records of the same money movements: a **bank statement** (the source of truth from the bank) and an **internal ledger** (what the bookkeeper or owner recorded). Reconciliation means proving these two records agree — line by line — and explaining every place they don't.

For a business with no dedicated accounting staff, this is usually done by hand: printing or exporting both records, and manually eyeballing rows in a spreadsheet to spot mismatches. It is repetitive, easy to get wrong, and grows harder every month as transaction volume increases.

Why mismatches happen

- Duplicate entries recorded on either side
- Timing differences — a cheque or transfer that clears days after it's logged
- Rounding or currency-conversion differences
- Missing transactions (bank fees, interest, or manual entries never logged)
- Genuine data-entry errors or fraud that needs a human's attention

Who feels this pain

- Solo founders and shopkeepers who do their own books at night
- Small accounting teams juggling many client ledgers at once
- Businesses in emerging markets without affordable dedicated accountants
- Anyone applying for a loan or credit line, where clean books are a prerequisite

2–8 hrs

spent monthly on manual reconciliation by a typical small business

90%+

of monthly transactions are usually routine, exact matches — yet still checked by hand

< 5%

of line items are typically genuine exceptions that actually need a human decision

Why this matters for financial inclusion

Accurate, up-to-date books are the gateway to formal credit, investor trust, and tax compliance. Small businesses that fall behind on reconciliation often present messy or stale financials — which quietly locks them out of loans and financial products that larger businesses take for granted. Automating the tedious 95% of reconciliation gives small business owners both their time back and a cleaner financial identity.



In short: reconciliation is a well-defined, repetitive comparison problem — compare two records, find where they disagree, decide why, and act. It is exactly the kind of structured, rules-plus-judgement task that an AI agent can safely automate for the routine cases, while escalating the genuinely ambiguous ones to a human.

SECTION 02

Proposed Solution

LedgerWright is an AI reconciliation agent that ingests a bank statement and an internal ledger, matches transactions automatically, explains every discrepancy in plain language, and only asks for human input where it's genuinely needed.

How it works, end to end

The business (or their accountant) uploads a bank statement and their ledger export — CSV, Excel, PDF, or a direct feed from tools like QuickBooks or Tally. LedgerWright normalizes both into a common transaction format, then runs them through a five-stage pipeline that mirrors how an experienced accountant reconciles: first find what obviously matches, then reason about what doesn't, then act on the confident cases, verify the totals still balance, and finally hand over a clean report with only the real exceptions flagged.

What makes it different from a spreadsheet

- Understands *why* two entries don't match, not just that they don't
- Auto-resolves the routine cases (duplicates, timing lags, rounding) with a visible audit trail
- Learns from every correction an accountant makes, improving future accuracy
- Turns a multi-hour manual task into a review of a handful of flagged items

Design principles

- Never force a match** — a match is only accepted once both running balances verify
- Human-in-the-loop by default** — low-confidence items always go to a review queue
- Explainability first** — every decision comes with a plain-English reason
- Auditable** — every auto-link is logged and reversible

Example: a single reconciliation run

Ledger Entry	Bank Entry	Diagnosis	Outcome
INV-2291 · ₹18,500 · 12 Aug	NEFT-77213 · ₹18,500 · 12 Aug	Exact amount + date match	Auto-matched
Vendor Pay · ₹9,000 · 03 Aug	UPI-44120 · ₹9,000 · 05 Aug	Same amount, 2-day clearing lag	Auto-matched
Office Supplies · ₹1,240.00	POS-9931 · ₹1,240.35	₹0.35 rounding / fee difference	Auto-matched
— (not in ledger)	Bank Charge · ₹150	Bank fee never logged internally	Flag: add entry
Consulting Fee · ₹42,000	— (no match found)	No corresponding bank entry at all	Flag: needs review

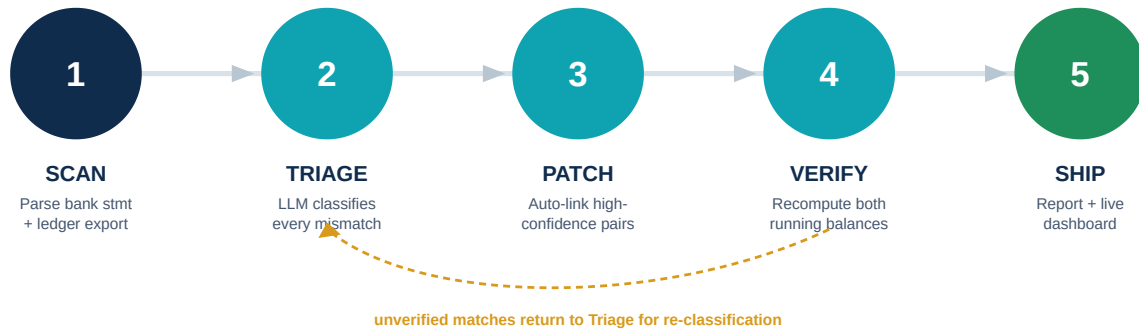


Outcome: in a batch of, say, 40 unmatched line items, LedgerWright typically auto-resolves the majority with a verified audit trail and surfaces only the handful of genuine exceptions — turning an hours-long manual task into a few minutes of focused review.

SECTION 03

System Architecture

A five-stage pipeline — Scan, Triage, Patch, Verify, Ship — that takes two raw financial records in and produces a verified reconciliation report out.



Layered system view

- **Ingestion layer** — CSV/Excel parser, PDF statement parser, OCR fallback for scanned statements
- **Matching engine** — rule-based + fuzzy pre-filter (amount, date window, reference string) before invoking the LLM, to keep cost and latency low
- **Reasoning layer** — LLM triages only genuinely ambiguous rows, with structured output (category + confidence + explanation)
- **Verification layer** — recomputes ledger vs. bank running totals after every auto-link; rejects matches that break balance integrity
- **Presentation layer** — Next.js dashboard + exportable PDF/Excel reconciliation report

Data flow summary

- Bank statement + ledger → normalized transaction table
- Fuzzy matcher removes the easy 1:1 exact matches first
- Remaining unmatched rows go to the LLM triage stage
- Confident classifications get auto-linked and verified
- Low-confidence / failed-verification rows enter the human review queue
- Final state renders on the dashboard and exports as a report

SECTION 04

Key Features

Nine capabilities that take reconciliation from a manual, hours-long chore to a guided few-minute review.

01

Multi-source ingestion

Accepts bank statements and ledger exports as CSV, Excel, or PDF, plus direct exports from QuickBooks / Tally / Zoho Books.

02

Automated transaction matching

Fuzzy-matches transactions on amount, date proximity, and reference text to instantly clear the routine, exact matches.

03

AI discrepancy triage

Classifies every remaining mismatch — duplicate, timing lag, rounding, missing entry, or genuine error — with a plain-English reason.

04

Auto-resolution engine

Automatically links high-confidence matches while keeping a full, reversible audit trail of every decision made.

05

Human-in-the-loop review

Routes only genuinely ambiguous or unverified items to a simple review queue — never forces a match silently.

06

Live reconciliation dashboard

Real-time view of matched, flagged, and unresolved items, with drill-down into any transaction's reasoning trail.

07

Exportable reports

One-click PDF or Excel reconciliation reports formatted for accountants, auditors, or loan applications.

08

Continuous learning

Every manual correction an accountant makes feeds back into the matcher, raising auto-match confidence over time.

09

Secure by design

Encrypted storage of financial data, JWT-based access control, and a full audit log of every automated action.



Net effect: what used to take 2–8 hours of manual line-by-line checking becomes a focused review of a handful of flagged exceptions.

SECTION 05

Technology Stack

A lean, cost-aware stack: cheap rule-based matching handles the bulk of transactions, and the LLM is reserved for genuinely ambiguous cases.

Application layer

Fx FastAPI Backend API & orchestration	Nx Next.js Reconciliation dashboard UI	Tw Tailwind CSS Dashboard styling
---	---	--

Intelligence & matching

Ge Gemini (LLM) Discrepancy triage & explanations	Rf RapidFuzz Fuzzy pre-matching engine	Pd pandas Transaction normalization
--	---	--

Ingestion & parsing

Pf pdfplumber PDF bank statement parsing	Oc pytesseract (OCR) Scanned statement fallback	Api REST connectors QuickBooks / Tally / Zoho exports
---	--	--

Data, infra & security

Pg PostgreSQL Transactions & audit trail	Dk Docker Containerized deployment	Jw JWT + encryption Auth & data-at-rest security
---	---	---



Why this stack: the fuzzy-matching pre-filter clears the majority of transactions without ever calling the LLM, so Gemini is only invoked on the small, genuinely ambiguous remainder — keeping the system fast and inexpensive to run even for high transaction volumes.

SECTION 06

Implementation Plan

A phased build plan for the hackathon, followed by a short post-hackathon roadmap toward a production-ready product.

Hackathon build plan

- PHASE 1**

Data ingestion & normalization

Build CSV/Excel/PDF parsers for bank statements and ledger exports; normalize both into a single transaction schema; assemble a small set of realistic sample datasets for testing.
- PHASE 2**

Matching engine (Scan + Triage)

Implement the rule-based / fuzzy pre-matcher for exact and near-exact matches; wire up the LLM triage stage for remaining mismatches with structured, explainable output.
- PHASE 3**

Auto-linking & verification (Patch + Verify)

Build the auto-link logic for high-confidence matches, the running-balance verification check, and the audit trail that logs every automated decision.
- PHASE 4**

Dashboard & reporting (Ship)

Ship the Next.js dashboard showing matched / flagged / unresolved items, a drill-down view per transaction, and one-click PDF / Excel report export.
- PHASE 5**

Testing, polish & demo prep

Run the full pipeline against sample statements end-to-end, tune confidence thresholds, fix edge cases, and prepare a live "before vs. after" demo for judges.

Post-hackathon roadmap

Short term

- Direct bank feed / API integrations
- Multi-currency support
- Configurable auto-match confidence thresholds

Mid term

- Accounting software plug-ins (QuickBooks, Tally, Zoho)
- Multi-entity reconciliation for accounting firms
- Scheduled, recurring auto-reconciliation runs

Long term

- Predictive cash-flow insights from reconciled data
- Bank-grade compliance certifications
- White-label offering for regional accounting firms



Demo goal: upload two sample statements live, show mismatches getting found and explained in real time, and land on a concrete "before: many unmatched line items → after: a handful flagged for human review" result.